```
#Task 1: Dataset Understanding:
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
```

```
from google.colab import files
uploaded = files.upload()

import pandas as pd

df = pd.read_csv('customer_churn_nn.csv')
```

```
Choose Files customer_churn_nn.csv
customer_churn_nn.csv(text/csv) - 168498 bytes, last modified: 5/17/2026 - 100% done
Saving customer_churn_nn.csv to customer_churn_nn.csv
```

```
df.head()
```

	customer_id	region	plan_type	contract_type	payment_method	tenure_months	monthly_charges_inr	avg_login_days_per_mon
0	CUST0001	South	Standard	Month-to-month	Debit Card	30	687.40	
1	CUST0002	West	Premium	Month-to-month	Wallet	15	1029.74	
2	CUST0003	Central	Standard	Month-to-month	Credit Card	72	732.07	
3	CUST0004	West	Premium	Month-to-month	Credit Card	22	959.51	
4	CUST0005	North	Premium	Month-to-month	Net Banking	11	890.20	

Next steps: Generate code with df | New interactive sheet

```
print('Number of Rows:', df.shape[0])
print('Number of Columns:', df.shape[1])
```

```
Number of Rows: 2000
Number of Columns: 17
```

```
print(df.dtypes)
```

```
customer_id                    object
region                         object
plan_type                      object
contract_type                  object
payment_method                 object
tenure_months                   int64
monthly_charges_inr           float64
avg_login_days_per_month        int64
support_tickets_last_90_days    int64
payment_delay_days              int64
data_usage_gb                 float64
satisfaction_score            float64
last_complaint_days_ago         int64
discount_percent                int64
autopay_enabled                 int64
referral_count                  int64
churn                           int64
dtype: object
```

```
print(df['churn'].value_counts())
```

```
churn
0    1969
1      31
Name: count, dtype: int64
```

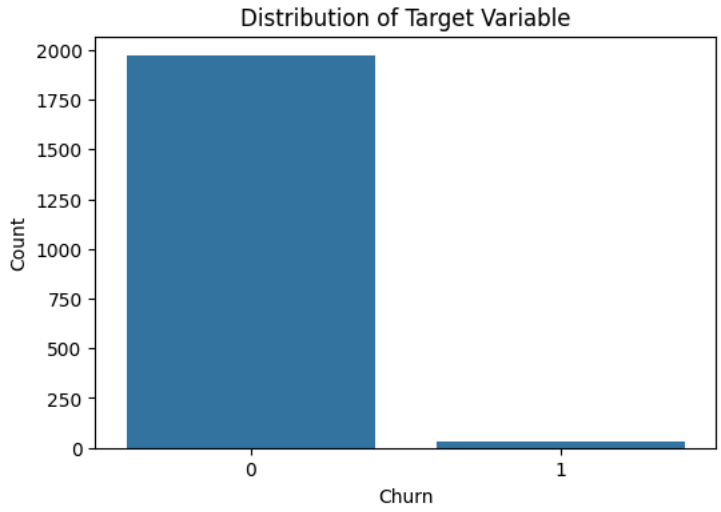
```
print(df.isnull().sum())
```

```
customer_id                    0
region                         0
plan_type                      0
contract_type                  0
payment_method                 0
tenure_months                  0
monthly_charges_inr            0
avg_login_days_per_month       0
support_tickets_last_90_days   0
payment_delay_days             0
data_usage_gb                  0
satisfaction_score             0
last_complaint_days_ago        0
discount_percent               0
autopay_enabled                0
referral_count                 0
churn                          0
dtype: int64
```

```
print(df.describe())
```

```
       customer_id       region    plan_type  contract_type  payment_method  \
count  2000.000000  2000.000000  2000.000000    2000.000000     2000.000000
mean    999.500000     2.003000     1.598000       0.588000        1.987500
std     577.494589     1.436671     1.279928       0.727682        1.426661
min       0.000000     0.000000     0.000000       0.000000        0.000000
25%     499.750000     1.000000     0.000000       0.000000        1.000000
50%     999.500000     2.000000     2.000000       0.000000        2.000000
75%    1499.250000     3.000000     3.000000       1.000000        3.000000
max    1999.000000     4.000000     3.000000       2.000000        4.000000

       tenure_months  monthly_charges_inr  avg_login_days_per_month  \
count    2000.000000          2000.000000               2000.000000
mean       25.362000           766.487295                 18.099000
std        14.128651           393.420070                  5.400628
min         1.000000           255.450000                  0.000000
25%        15.000000           427.782500                 15.000000
50%        23.000000           688.355000                 18.000000
75%        33.000000          1007.372500                 22.000000
max        72.000000          2156.520000                 30.000000

       support_tickets_last_90_days  payment_delay_days  data_usage_gb  \
count                   2000.000000         2000.000000    2000.000000
mean                       1.953000            3.555000      90.007625
std                        1.463852            3.885682      53.215719
min                        0.000000            0.000000       0.500000
25%                        1.000000            1.000000      51.777500
50%                        2.000000            2.000000      80.245000
75%                        3.000000            5.000000     119.097500
max                        8.000000           31.000000     265.510000

       satisfaction_score  last_complaint_days_ago  discount_percent  \
count          2000.00000              2000.000000       2000.000000
mean              6.87395                46.616500          8.255000
std               1.52428                55.065775          7.553708
min               1.00000                 0.000000          0.000000
25%               5.87500                 6.000000          0.000000
50%               6.80000                28.500000          5.000000
75%               8.00000                68.000000         15.000000
max              10.00000               424.000000         20.000000

       autopay_enabled  referral_count        churn
count      2000.000000     2000.000000  2000.000000
mean          0.597500        0.918000     0.015500
std           0.490524        1.041546     0.123561
min           0.000000        0.000000     0.000000
25%           0.000000        0.000000     0.000000
50%           1.000000        1.000000     0.000000
75%           1.000000        1.000000     0.000000
max           1.000000        7.000000     1.000000
```

```
plt.figure(figsize=(6,4))
sns.countplot(x='churn', data=df)
plt.title('Distribution of Target Variable')
plt.xlabel('Churn')
plt.ylabel('Count')
plt.show()
```




```
#Task 2: Data Preprocessing
```

```
categorical_cols = df.select_dtypes(include=['object']).columns

label_encoders = {}

for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le
```

```
X = df.drop('churn', axis=1)
y = df['churn']
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

```
#Task 3: Neural Network Model Building
```

```
model = Sequential()

# Input Layer + Hidden Layer
model.add(Dense(16, input_dim=X_train.shape[1], activation='relu'))

# Hidden Layer
model.add(Dense(8, activation='relu'))

# Output Layer
model.add(Dense(1, activation='sigmoid'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(optimizer=Adam(learning_rate=0.001), loss='binary_crossentropy', metrics=['accuracy'])
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	272
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 1)	9

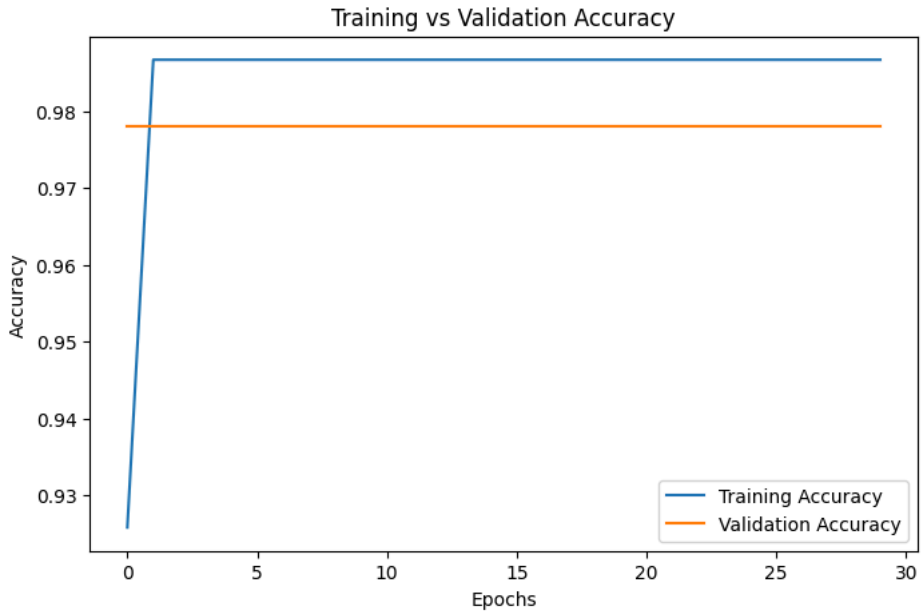
Total params: 417 (1.63 KB)
Trainable params: 417 (1.63 KB)

```
#Task 4: Training and Evaluation
```

```
history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 2/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9867 - loss: 0.2197 - val_accuracy: 0.9781 - val_loss: 0.1717
Epoch 3/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.1299 - val_accuracy: 0.9781 - val_loss: 0.1257
Epoch 4/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0940 - val_accuracy: 0.9781 - val_loss: 0.1103
Epoch 5/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9867 - loss: 0.0792 - val_accuracy: 0.9781 - val_loss: 0.1044
Epoch 6/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9867 - loss: 0.0722 - val_accuracy: 0.9781 - val_loss: 0.1011
Epoch 7/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0676 - val_accuracy: 0.9781 - val_loss: 0.0987
Epoch 8/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9867 - loss: 0.0644 - val_accuracy: 0.9781 - val_loss: 0.0968
Epoch 9/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0623 - val_accuracy: 0.9781 - val_loss: 0.0965
Epoch 10/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9867 - loss: 0.0598 - val_accuracy: 0.9781 - val_loss: 0.0949
Epoch 11/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9867 - loss: 0.0577 - val_accuracy: 0.9781 - val_loss: 0.0936
Epoch 12/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9867 - loss: 0.0561 - val_accuracy: 0.9781 - val_loss: 0.0940
Epoch 13/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0544 - val_accuracy: 0.9781 - val_loss: 0.0930
Epoch 14/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9867 - loss: 0.0533 - val_accuracy: 0.9781 - val_loss: 0.0934
Epoch 15/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9867 - loss: 0.0516 - val_accuracy: 0.9781 - val_loss: 0.0926
Epoch 16/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9867 - loss: 0.0506 - val_accuracy: 0.9781 - val_loss: 0.0922
Epoch 17/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9867 - loss: 0.0492 - val_accuracy: 0.9781 - val_loss: 0.0922
Epoch 18/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9867 - loss: 0.0482 - val_accuracy: 0.9781 - val_loss: 0.0925
Epoch 19/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9867 - loss: 0.0470 - val_accuracy: 0.9781 - val_loss: 0.0921
Epoch 20/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0461 - val_accuracy: 0.9781 - val_loss: 0.0923
Epoch 21/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0448 - val_accuracy: 0.9781 - val_loss: 0.0918
Epoch 22/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0439 - val_accuracy: 0.9781 - val_loss: 0.0919
Epoch 23/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0431 - val_accuracy: 0.9781 - val_loss: 0.0922
Epoch 24/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0421 - val_accuracy: 0.9781 - val_loss: 0.0920
Epoch 25/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0411 - val_accuracy: 0.9781 - val_loss: 0.0917
Epoch 26/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0404 - val_accuracy: 0.9781 - val_loss: 0.0919
Epoch 27/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0394 - val_accuracy: 0.9781 - val_loss: 0.0926
Epoch 28/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0384 - val_accuracy: 0.9781 - val_loss: 0.0929
Epoch 29/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0376 - val_accuracy: 0.9781 - val_loss: 0.0933
Epoch 30/30
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9867 - loss: 0.0368 - val_accuracy: 0.9781 - val_loss: 0.0938
```

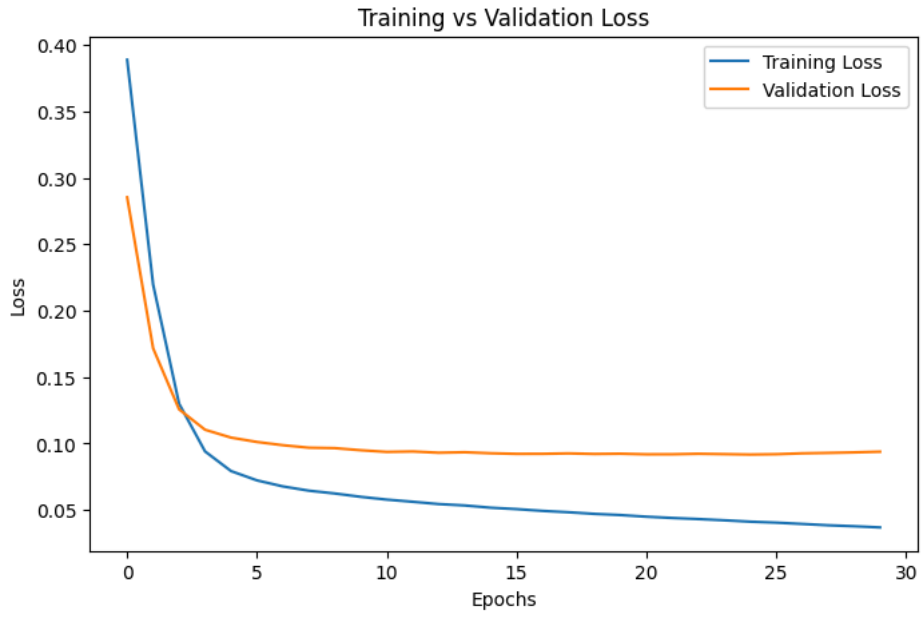
```
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training vs Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```




```
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training vs Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show
```

```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None
```

```
**Auto-show in jupyter notebooks**

The jupyter backends (activated via ``%matplotlib inline``,
``%matplotlib notebook``, or ``%matplotlib widget``), call ``show()`` at
the end of every cell by default. Thus, you usually don't have to call it
explicitly there.
```

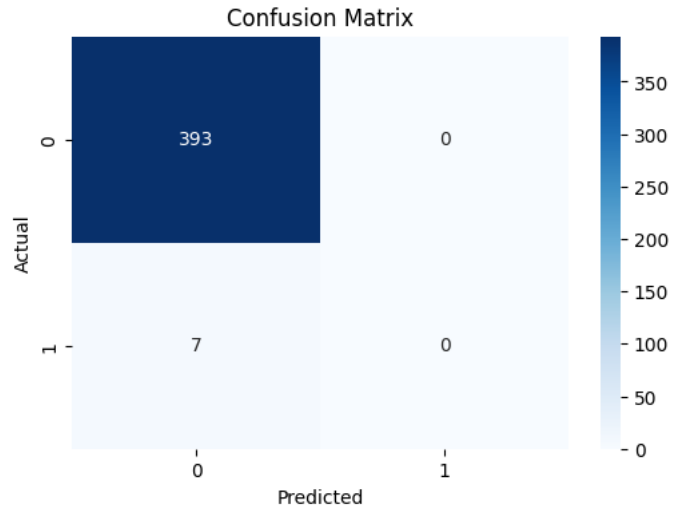



```
loss, accuracy = model.evaluate(X_test, y_test)

print('Test Loss:', loss)
print('Test Accuracy:', accuracy)

y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5).astype(int)
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9825 - loss: 0.0824
Test Loss: 0.08240415900945663
```

```
Test Accuracy: 0.9825000166893005
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step
```

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

print(classification_report(y_test, y_pred))
```




```
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       393
           1       0.00      0.00      0.00         7

    accuracy                           0.98       400
   macro avg       0.49      0.50      0.50       400
weighted avg       0.97      0.98      0.97       400

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
#Task 5: Hyperparameter Experimentation
```

```
#Experiment 1

model_1 = Sequential([
    Dense(16, activation='relu', input_dim=X_train.shape[1]),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])

model_1.compile(optimizer=Adam(0.001), loss='binary_crossentropy', metrics=['accuracy'])

history_1 = model_1.fit(X_train, y_train, epochs=30, batch_size=32, verbose=0)

loss_1, acc_1 = model_1.evaluate(X_test, y_test, verbose=0)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
#Experiment 2

model_2 = Sequential([
    Dense(32, activation='relu', input_dim=X_train.shape[1]),
    Dense(16, activation='relu'),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])

model_2.compile(optimizer=Adam(0.001), loss='binary_crossentropy', metrics=['accuracy'])
```

```
history_2 = model_2.fit(X_train, y_train, epochs=50, batch_size=16, verbose=0)

loss_2, acc_2 = model_2.evaluate(X_test, y_test, verbose=0)
```

```
#Experiment 3

model_3 = Sequential([
    Dense(16, activation='tanh', input_dim=X_train.shape[1]),
    Dense(8, activation='tanh'),
    Dense(1, activation='sigmoid')
])

model_3.compile(optimizer=Adam(0.01), loss='binary_crossentropy', metrics=['accuracy'])

history_3 = model_3.fit(X_train, y_train, epochs=20, batch_size=64, verbose=0)

loss_3, acc_3 = model_3.evaluate(X_test, y_test, verbose=0)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
#Comparison Table

comparison_df = pd.DataFrame({
    'Experiment': ['Experiment 1', 'Experiment 2', 'Experiment 3'],
    'Hidden Layers': [2, 3, 2],
    'Neurons': ['16-8', '32-16-8', '16-8'],
    'Activation': ['ReLU', 'ReLU', 'Tanh'],
    'Learning Rate': [0.001, 0.001, 0.01],
    'Batch Size': [32, 16, 64],
    'Epochs': [30, 50, 20],
    'Test Accuracy': [acc_1, acc_2, acc_3]
})

print(comparison_df)
```

```
     Experiment  Hidden Layers  Neurons Activation  Learning Rate  Batch Size  \
0  Experiment 1              2     16-8       ReLU          0.001          32
1  Experiment 2              3  32-16-8       ReLU          0.001          16
2  Experiment 3              2     16-8       Tanh          0.010          64

   Epochs  Test Accuracy
0      30         0.9825
1      50         0.9800
2      20         0.9725
```

```
import os

# Create the 'results' directory if it doesn't exist
output_dir = 'results'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)
```